ASSIGNMENT 2

Group MobyDick

Fine-Grained Food Classification:
Custom CNN vs. Two-Stage Pretrained ResNet-50 on Food-101

CO3133 Deep Learning and Its Applications · Semester 261 · Instructor: Lê Thành Sách

OUR TEAM

1

Quách Kiến Long

2352698

Data audit, stratified split & EDA

2

Lê Quốc Bình

2352117

Training loop, AMP & compute-cost logging

3

Lê Thanh Lâm

2252421

Learning curves & Grad-CAM

4

Tô Nhã Vy

2353360

Qualitative & error analysis, report

THE PROBLEM

Task

Single-label, fine-grained classification of food photos into 10 Food-101 dishes.

Input: one RGB photo, resized to 3×224×224.

Output: 10 logits » one dish label.

Loss: cross-entropy. Metrics: accuracy & macro-F1.

Research question

How much do ImageNet pretraining and full fine-tuning gain over a CNN trained from scratch, and at what compute cost?

Why it is hard

Dishes share colours and textures (baked desserts, raw red meat), plates are cluttered, lighting varies, and training labels are noisy.

DATASET & EDA

Food-101, 10-class subset

Source: official ETH Zurich release (Bossard et al., 2014), archive SHA-256 recorded.

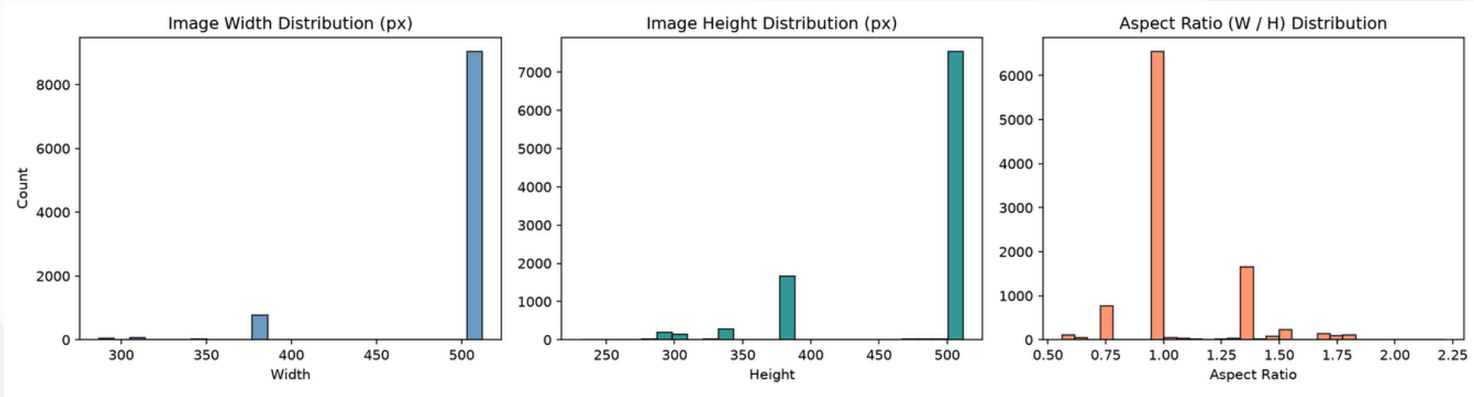
Classes: first 10 labels A–Z, fixed before experiments.

Balance: 1,000 per class (750 train / 250 test).

Size: 287–512 × 231–512 px, median 512 × 512; aspect 0.56–2.22.

Channels: 9,999 RGB + 1 grayscale » RGB.





INTEGRITY & SPLIT

1

Audit before splitting

All 10,000 images open in PIL; SHA-256 found 0 exact duplicates; 64-bit aHash flagged 17 near-duplicate pairs (kept, logged in perceptual_duplicates.csv).

2

Stratified split, seed 42

Official 7,500 train » 6,750 train / 750 val (675 / 75 per class). Unit = one image record, saved in split_manifest.csv and reused by every run.

3

Test stays untouched

2,500 official test images (250 per class) are used once, after checkpoint selection on validation macro-F1.

DATA PIPELINE

Training branch

RandomResizedCrop(224, scale 0.7–1.0)

RandomHorizontalFlip(p = 0.5)

ColorJitter(0.2, 0.2, 0.2, 0.05)

Normalize with ImageNet mean / std

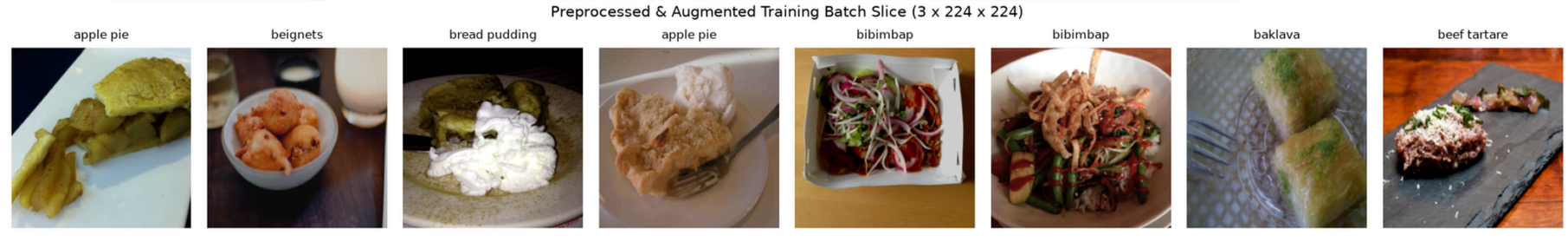
Validation / test branch

Resize(256) » CenterCrop(224)

Normalize with ImageNet mean / std

Deterministic, no gradient updates

Batch 64 » tensor 64×3×224×224



MODELS

Baseline: Custom CNN

Self-implemented, trained from scratch.
4 blocks × (2 × Conv3×3 + BN + ReLU) + MaxPool; channels 32 » 64 » 128 » 256; 224 » 14 px.
Global average pool » FC 256 » BN » ReLU » Dropout 0.3 » 10.
Params: 1,242,090 (all trainable).

Pretrained: ResNet-50

torchvision ImageNet-1K V1 weights (He et al., 2016).
Original classifier replaced by a new 2048 » 10 linear head.
Residual blocks give a deep, well-conditioned feature extractor learned from 1.28 M images.
Params: 23,528,522 total; head only 20,490.

CNN TRAINING STEP

STEP 3 **Custom CNN Architecture**

INPUT: **Food-101 RGB Image (224 x 224 x 3)**

Block 1 **32 Filters** 224x224 → 112x112
2x [Conv3x3 + BatchNorm + ReLU] + MaxPool

Block 2 **64 Filters** 112x112 → 56x56
2x [Conv3x3 + BatchNorm + ReLU] + MaxPool

Block 3 **128 Filters** 56x56 → 28x28
2x [Conv3x3 + BatchNorm + ReLU] + MaxPool

Block 4 **256 Filters** 28x28 → 14x14
2x [Conv3x3 + BatchNorm + ReLU] + MaxPool

Classification Head **10 Food Classes**
Global AvgPool → Linear(256) → Dropout(0.3) → 10 Food Logits

Total Size: 1.24 Million Parameters (100% Trainable)

STEP 4 **Training Setup (From Scratch)**

1. Initialization
Trained 100% from scratch (no pretrained weights)

2. Loss Function
Cross-Entropy Loss

3. Optimizer
AdamW (learning rate = 0.001)

4. Training Schedule
15 Epochs (~3.5 minutes on GPU)

✓ **Fair Baseline: Same data & 15 epochs as ResNet-50**

Setup 1: ResNet-50 Frozen Backbone

Input:

Food-101 Image (224*224*3)

Frozen Backbone:

- Was pre-trained.
- Conv1 + Maxpool (56*56)
- Layer 1 - 4 (16 Bottelnecks)
- 25,508,032 lock params (99.1%)

Linear Layer: 2048 → 10

- Map 2,048 ImageNet features to 10 food classes
- 20,490 trainable active params (0.09%)

Setup 2: ResNet-50 FINE-TUNING PROCEDURE

1

Stage 1 · epochs 1–5: head only

Backbone frozen; train the new head (20,490 params) with AdamW, lr 1e-3, weight decay 1e-4.

2

Stage 2 · epochs 6–15: full fine-tune

Unfreeze all 23.5 M params; differential LR: backbone 1e-4, head 5e-4, so pretrained features move slowly.

3

Shared rules

Mixed precision (AMP + GradScaler), batch 64, 15 epochs; keep the epoch with the best validation macro-F1.

CONTROLLED EXPERIMENT

Hypothesis Full fine-tuning improves macro-F1 over a frozen backbone, at extra training cost.

Changed factor Frozen backbone (head only, 15 epochs) vs. two-stage full fine-tuning.

Fixed factors Same manifest, seed 42, 224 px, augmentation, AdamW, batch 64, 15 epochs, AMP, checkpoint rule.

Metrics & decision Accuracy, macro-F1 + params, train time, latency, VRAM. Select on validation macro-F1; report test once.

Ablation Full fine-tune with vs. without training augmentation (all else fixed).

RTX 5060 Laptop (8 GB) | Core Ultra 9 275HX | 32 GB | WSL2 | Python 3.13.13 | PyTorch 2.12.1+cu130

RESULTS

Model	Trainable params	Best epoch	Val macro-F1	Test acc	Test macro-F1
Custom CNN	1,242,090	15	0.6180	63.36%	0.6335
ResNet-50 frozen	20,490	8	0.7653	80.24%	0.8022
ResNet-50 full FT	**23,528,522**	**14**	**0.8673**	**88.80%**	**0.8877**

+8.6 pts

macro-F1, full FT vs. frozen

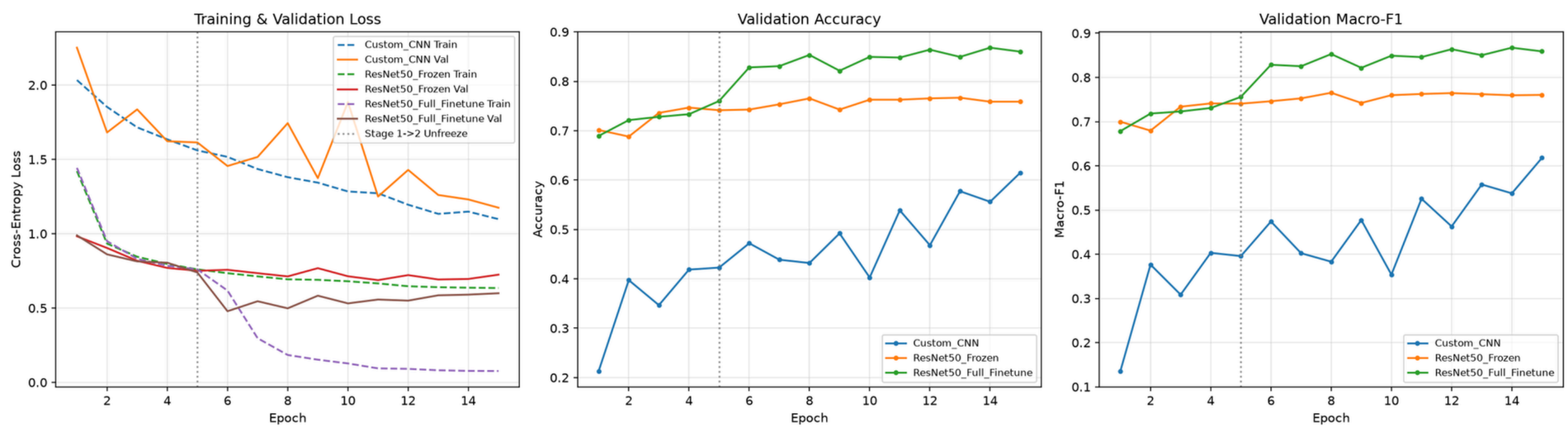
Hypothesis supported

+25.4 pts

macro-F1, full FT vs. Custom CNN

Pretraining is the biggest gain

LEARNING CURVES



Unfreeze jump

Full FT val accuracy rises from ≈0.76 to ≈0.83 right after the stage switch at epoch 5.

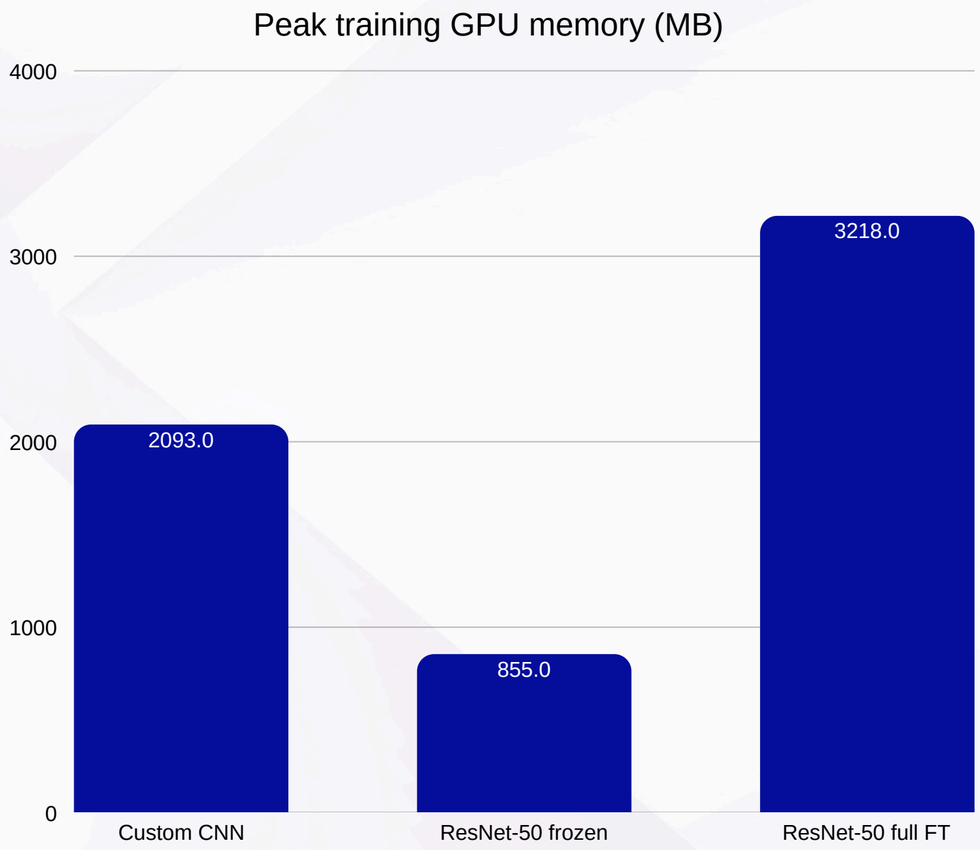
Frozen plateau

Frozen head stalls near 0.76 from epoch ≈4; best epoch 8.

Scratch under-trained

Custom CNN still climbing at epoch 15 (best = last epoch); val loss is noisy.

COMPUTE COST



	Train	Latency	VRAM
Custom CNN	219 s	0.89 ms	2.0 GB
ResNet-50 frozen	223 s	1.43 ms	0.8 GB
ResNet-50 full FT	**268 s**	**1.44 ms**	**3.1 GB**

Reading the cost

Full FT: +20% training time and 3.8× the memory of the frozen run.
Inference is the same network, so latency is equal (≈1.44 ms/img, batch 64).
Custom CNN is fastest at inference (0.89 ms/img) but 25 pts worse.

AUGMENTATION ABLATION

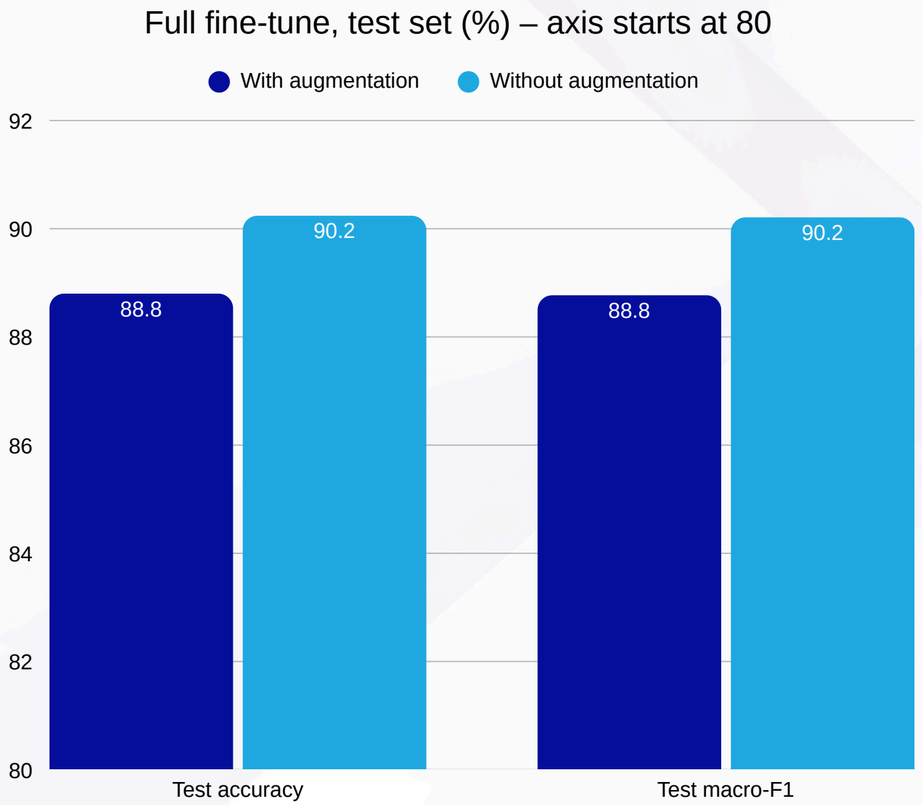
With vs. without augmentation

Same two-stage fine-tune; only the training transforms change.

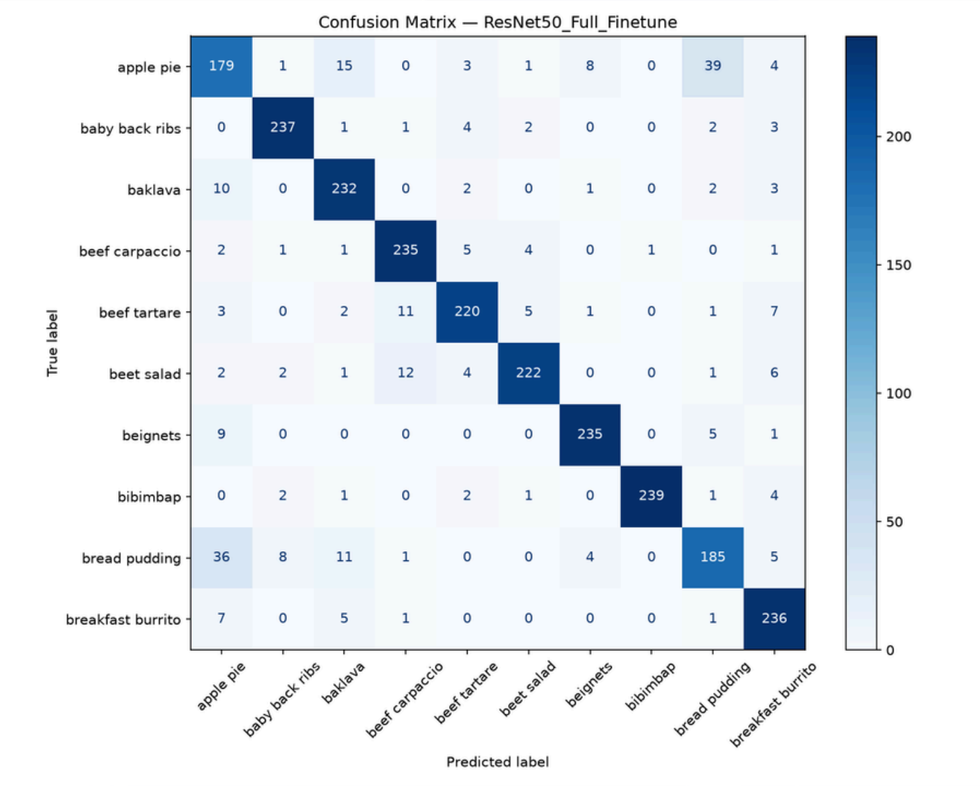
Without aug: 90.24% / 0.9021, val F1 0.8721, 230 s.

With aug: 88.80% / 0.8877, val F1 0.8673, 268 s.

Finding: augmentation did not help in 15 epochs (-1.4 pts F1); one seed, so the gap may be within run-to-run noise.



ERROR ANALYSIS



Full FT: 280 test errors

Baked desserts: apple pie » bread pudding 39, bread pudding » apple pie 36, apple pie » baklava 15.

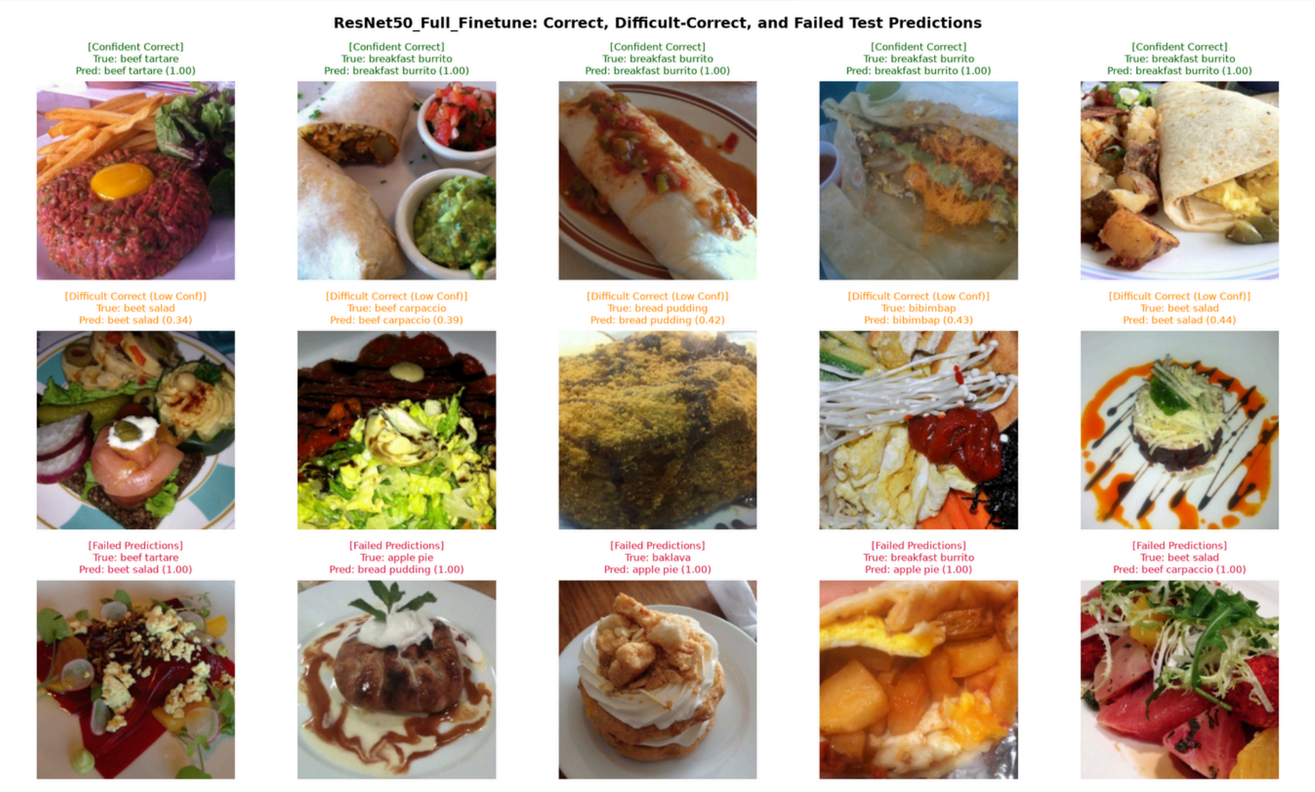
Raw red dishes: beet salad » beef carpaccio 12, beef tartare » beef carpaccio 11.

Clutter / lighting: burrito-like plates, dark glaze.

Taxonomy: top-15 pairs = 189 errors: 128 desserts, 33 red dishes, 20 clutter, 8 lighting.

Weakest class: apple pie (F1 0.72).

QUALITATIVE RESULTS



Three tiers

Confident correct: clear burritos & tartare, p = 1.00.

Difficult correct: busy salads, bibimbap, p = 0.34–0.44.

Failed: all five wrong with p = 1.00 » the model is over-confident on look-alikes.

GRAD-CAM

Where does each model look?

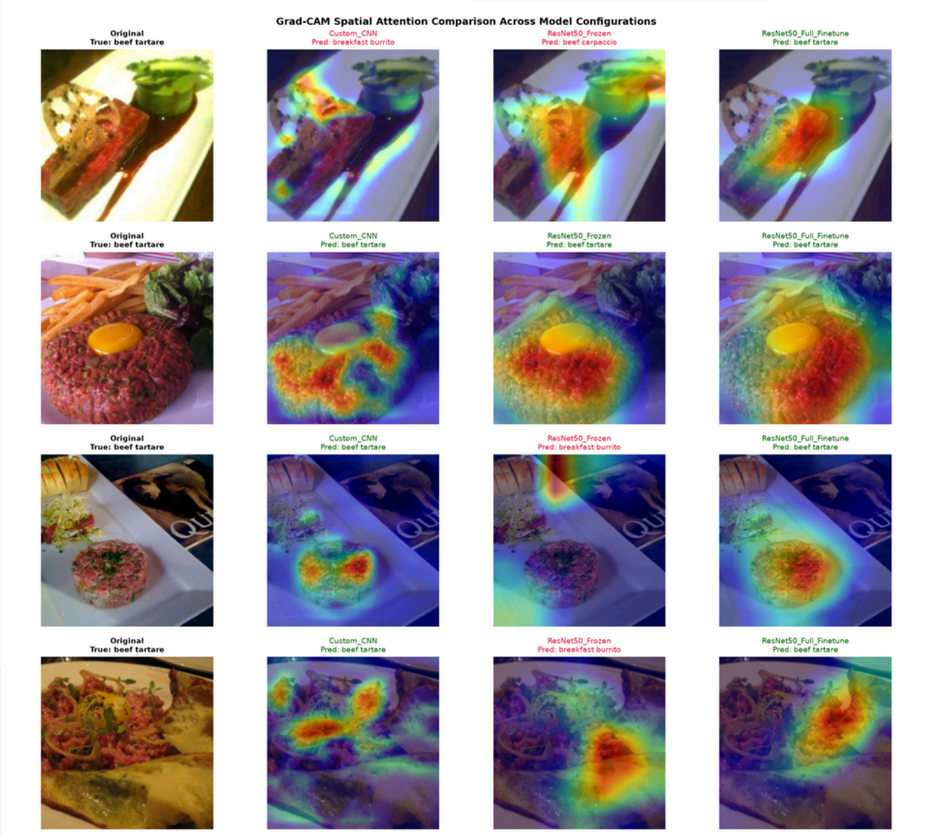
Four beef-tartare test images, last conv block of each model.

Full FT: 4 / 4 correct, attention centred on the meat.

Frozen: 3 / 4 wrong (carpaccio, burrito ×2), heat drifts to sauce, bread and background.

Custom CNN: 1 / 4 wrong, scattered blobs.

Only 4 images of one class: illustrative, not a statistic.



CONCLUSION

Takeaways

Pretraining is decisive: 88.8% vs. 63.4% from scratch.

Unfreezing adds 8.6 pts macro-F1 for +20% train time and 3.8× memory, with no inference cost.

Remaining errors are look-alike dishes, mostly baked desserts.

Augmentation did not help in our 15-epoch run.

Limitations & next

Single seed and run; 10-class subset only.

17 near-duplicate pairs kept (3 span train–test).

Custom CNN under-trained; no per-epoch logs or checkpoint hashes retained.

Next: multi-seed mean ± std, longer schedules, de-duplication, calibration.

THANK YOU